

Technical Evaluation Task: Async Job Processing Platform

Context

We are evaluating candidates for a production engineering role involving FastAPI, Redis/BullMQ-style async processing, React dashboards, Docker, API security, and production hardening.

This is a standalone evaluation task. You will not receive access to our production codebase. The goal is to demonstrate how you design, build, document, and harden a small production-style system from an empty repository.

Estimated effort: 5–10 hours.

Objective

Build a small async job processing platform.

The system should allow a user to submit a job through an API or dashboard, process the job asynchronously, track its status, and display real-time or near-real-time updates in a frontend dashboard.

Required Stack

You may use reasonable equivalents, but the **preferred stack** is:

- Backend: Python FastAPI
 - Queue/state: Redis
 - Worker: Python or Node.js worker
 - Frontend: React/Vite
 - Deployment: Docker Compose
 - Database: Postgres preferred, Redis-only acceptable if clearly documented
-

Required Features

1. Backend API

Implement the following endpoints:

```
GET /health
POST /jobs
GET /jobs/{job_id}
GET /jobs
```

Expected behavior

POST /jobs should accept a JSON payload such as:

```
{
  "url": "https://example.com",
  "job_type": "extract"
}
```

When submitted, the job should be queued and return immediately with:

```
{
  "job_id": "...",
  "status": "queued"
}
```

GET /jobs/{job_id} should return:

```
{
  "job_id": "...",
  "status": "queued|processing|completed|failed",
  "result": {},
  "error": null,
  "created_at": "...",
  "updated_at": "..."
}
```

2. Async Worker

Create a worker process that consumes jobs from the queue.

The worker should:

- Pick up queued jobs.
- Mark jobs as `processing`.
- Simulate processing for 5–15 seconds.
- Randomly fail a small percentage of jobs.

- Mark jobs as `completed` or `failed`.
- Store a simple result object for completed jobs.

Example completed result:

```
{  
  "title": "Example Domain",  
  "summary": "Fake extracted content",  
  "processed_by": "worker-1"  
}
```

3. Frontend Dashboard

Build a simple React/Vite dashboard.

The dashboard should support:

- Submit a new job.
- View recent jobs.
- View job status.
- View result/error.
- Automatically update job status without requiring a full page refresh.

You may use either:

- Server-Sent Events,
- WebSockets,
- polling every few seconds.

If you choose polling, explain why.

Visual design is not the priority. Functionality, correctness, and maintainability matter more than styling.

4. Security / Hardening

Add basic production-minded protections:

- API key authentication for job endpoints.
- Basic rate limiting or clear explanation of how you would implement it.
- Environment-based configuration.
- No hardcoded secrets.
- CORS configuration.
- Basic request validation.

- Structured logging or clear operational logs.
-

5. Docker / Deployment

Provide a working Docker Compose setup.

It should start the full stack with one command:

```
docker compose up --build
```

Include:

- Backend service
- Worker service
- Redis service
- Frontend service
- Postgres service if used

Add reasonable health checks where possible.

Required Documentation

Your README must include:

1. Setup instructions.
 2. Architecture overview.
 3. API usage examples.
 4. How the async job flow works.
 5. How to run tests or manually validate the system.
 6. Security notes.
 7. Known limitations.
 8. What you would improve with more time.
 9. Backup/recovery strategy for Redis/Postgres.
 10. Production deployment considerations.
-

Boundaries

You may configure the VM at the application level.

You may install:

- Docker
- Docker Compose
- Node.js
- Python
- Git
- required application dependencies

You must not modify:

- host networking,
- firewall rules,
- DNS,
- routing,
- kernel-level settings,
- or anything unrelated to the application stack.

If you believe a host-level change is needed, document it instead of applying it.

Deliverables

Please provide:

1. Git repository with your code.
 2. Clear README.
 3. Docker Compose file.
 4. `.env.example` file.
 5. Short demo video or screen recording, 5–10 minutes.
 6. Brief architecture notes explaining your decisions.
 7. Any assumptions or limitations.
-

Evaluation Criteria

We will evaluate:

- Correctness: Does it work?
- Architecture: Is it clean and maintainable?
- Async processing: Is the queue/worker flow reliable?
- Frontend/backend integration: Does the dashboard reflect job status correctly?
- Security awareness: Did you handle auth/config/CORS/rate limiting responsibly?

- Production thinking: Did you consider retries, failures, health checks, logging, backup, and recovery?
 - Simplicity: Did you avoid unnecessary overengineering?
 - Documentation: Can another engineer understand and run it?
 - Git hygiene: Are commits clear and organized?
 - Communication: Can you explain your tradeoffs?
-

Time Expectation

This task is designed to take approximately 5–10 hours.

Please prioritize a working, maintainable implementation over a visually polished interface.